A3-PS013-TC479

Incident Timeline and Evidence Collection for Small Teams

ARIXA Forensic Timeline: SHA-256 evidence integrity and chain of custody, UTC normalisation with time-zone, DST and clock-skew correction, and MITRE ATT&CK-mapped detection

Komal Ganjave : Team Lead
Komalganjave32@gmail.com

Yash Bhosale : Team Member
yashbhosale2403@gmail.com

Bhagyashri Thorat : Faculty Member
bhagyashrit@isquareit.edu.in

AIORI Hackathon 3 · Problem Statement PS-013
October 14, 2026

Reconstructed incident timeline (UTC)

ARIXA output · sample scenario 1 · SSH brute force

21:01:07Z **T1110 · HIGH**
Brute force: 15 failed SSH logins from 203.0.113.45

21:03:20Z **T1078 · CRITICAL**
Login as 'deploy' right after the failures

21:03:20Z **T1548 · HIGH**
sudo used after the suspicious login

21:05:02Z **T1003.008 · HIGH**
Password hashes read (/etc/shadow)

21:06:40Z **T1136.001 · HIGH**
Local account created (useradd)

Problem Description

Small teams must rebuild “what happened, and when” from scattered logs, with no SIEM, no forensic lab and no room for error

Fragmented evidence
Logs sit on many servers (SSH bastion, web, database) with no single view

Different log formats
Linux auth.log (syslog) and Nginx/Apache access.log use different fields

Ambiguous time
Syslog (RFC 3164) has no year or zone; servers use different zones and DST

Clock skew
Unsynchronised clocks drift by minutes, enough to reverse an attack’s order

No evidence integrity
Without hashing and a chain of custody, findings can be challenged

Silent data loss, slow triage
Unreadable lines are skipped unnoticed; manual work takes ~45 min

Same attack, two servers, two clocks (scenario 3)

web01 · Asia/Kolkata (UTC+05:30)

```
[09/Oct/2026:14:02:10 +0530] "GET
/static/img/avatar.php?cmd=id"
```

db01 · America/New_York · clock 3 min slow

```
Oct  9 04:30:05 db01 sshd: Accepted password for
dbadmin
```

Read as written, the login (04:30) looks **9 h 32 min BEFORE** the web shell (14:02). The attacker’s pivot from web01 to db01 is invisible, and converting by hand is slow and error-prone.

Objective (PS-013): preserve evidence (SHA-256, custody), parse two log formats, build one UTC timeline, flag attacks with ATT&CK, report, log every unreadable line.
Keywords: DFIR, incident timeline, evidence collection, chain of custody, SHA-256, log analysis, UTC, DST, clock skew, MITRE ATT&CK, SOC, small teams

Background

Incident response needs one correct timeline: NIST defines the forensic process [1], but syslog has no year or zone and clocks drift

Forensic process (NIST SP 800-86) mapped to ARIXA modules [1]

Collection	Examination	Analysis	Reporting
evidence.py: SHA-256, read-only copy, manifest, custody log	parsers.py + normalize.py: fields, UTC time, parsing gaps	detect.py + attack_map.py: rules R1–R8, kill chain, graph	report.py: Markdown / PDF report, case bundle ZIP

Why time breaks investigations

- **No year, no zone:** RFC 3164 syslog writes “Oct 9 04:30:05” [6]; RFC 5424 adds RFC 3339 time [7][8]
- **Mixed sources:** access logs carry an offset (+0530); syslog lines do not
- **Daylight saving:** America/New_York is UTC−5 or UTC−4; use IANA zone names [10]
- **Clock drift:** without NTP clocks drift [9]; RFC 3227 says record clock vs UTC [4]
- **Real impact:** time errors have misled real investigations [15][16]
- **IR lifecycle [2]:** ARIXA serves detection & analysis and the post-incident report

Case study, scenario 3: db01 login vs web01 web shell ● shell ● login

1 Raw local time — MISSED

`db01 04:30:05 vs web01 14:02:10`

Login 9 h 32 min BEFORE the web shell



2 UTC, no skew fix — MISSED

`08:30:05Z vs 08:32:10Z`

Login 2 min 5 s BEFORE the web shell



3 UTC + DST + 180 s skew fix — DETECTED

`08:33:05Z vs 08:32:40Z`

Login 25 s AFTER: R6 lateral movement (T1021.004)



Literature & RFC Review

Standards define how evidence is kept and how time is written; existing tools are too heavy for a two-person team

Standard	Key guidance → how ARIXA applies it
NIST SP 800-86 [1]	Four forensic phases → pipeline mirrors collection, examination, analysis, reporting
NIST SP 800-61r2 [2], 800-92 [3]	Incident handling, log management → timeline + rules for detection and analysis
RFC 3227 [4], ISO/IEC 27037 [5]	Chain of custody, never alter evidence → read-only copies, custody log, SHA-256 manifest
RFC 3164 [6], RFC 5424 [7]	Syslog lacks year and zone → year inference + IANA zone; RFC 5424 lines logged as gaps
RFC 3339 [8], RFC 6557 [10]	UTC "Z" timestamps, DST rules → events stored as …Z; zone names, never fixed offsets
RFC 5905, NTP [9]	Unsynchronised clocks drift → per-source skew (e.g. +180 s) before correlation
FIPS 180-4 [11]	SHA-256 → hash at ingest, re-hash on verify (OK / MISMATCH)
MITRE ATT&CK [13], Kill Chain [14]	Common attack language → technique ID on every alert; 8-tactic kill chain
RFC 9110 [12]	HTTP status codes → 404-burst scan rule, 200-to-odd-path exploit rule

Research: *clock errors misled real cases [15]; timestamp correlation reveals skew [16]; automated timeline reconstruction [17]; super timelines [18]; real log datasets [19].*

Existing approaches vs a small team

Manual (spreadsheet, grep)

Free, but slow, error-prone and no integrity record

Plaso / log2timeline [18][23]

Huge format coverage; CLI-heavy, needs expertise

Timesketch [22]

Collaborative; needs Docker, OpenSearch, PostgreSQL

SIEM (Splunk, Elastic)

Real-time at scale; licence and infrastructure cost

Gap → ARIXA: one lightweight, explainable workflow with evidence integrity, skew-aware UTC, gap accounting and ATT&CK detection.

Proposed Solution

ARIXA Forensic Timeline: read-only pipeline, 8 ATT&CK-mapped rules, interactive dashboard, optional guarded AI

1 Collect & hash
SHA-256, read-only copy, custody log

2 Parse
auth.log + access.log, gaps with reason

3 Normalise
IANA zone, DST, skew → UTC

4 Store
SQLite timeline, indexed by time

5 Detect & map
Rules R1–R8 + MITRE ATT&CK

6 Report
Dashboard, PDF report, case ZIP

Rule	Detects	MITRE ATT&CK
R1	Brute force: ≥ 5 failed SSH logins per IP in 5 min	T1110
R2	Successful login after the failures (same IP, 10 min)	T1078
R3	Web scan: ≥ 10 HTTP 404s on ≥ 8 paths in 5 min	T1595
R4	Scan, then HTTP 200 to an unusual path (30 min)	T1190
R5	SQL injection, path traversal, web shell	T1190, T1505.003
R6	SSH login ≤ 15 min after a web shell on another host	T1021.004
R7	sudo ≤ 30 min after a suspicious login	T1548
R8	Risky sudo: shadow, useradd, SSH key, cron, SUID, DB dump	T1003.008, T1136.001, T1098.004, T1053.003, T1548.001, T1005

Every alert cites its event IDs and raw lines. Optional Gemini copilot: redacted prompts, custody-logged, cited IDs verified [20].

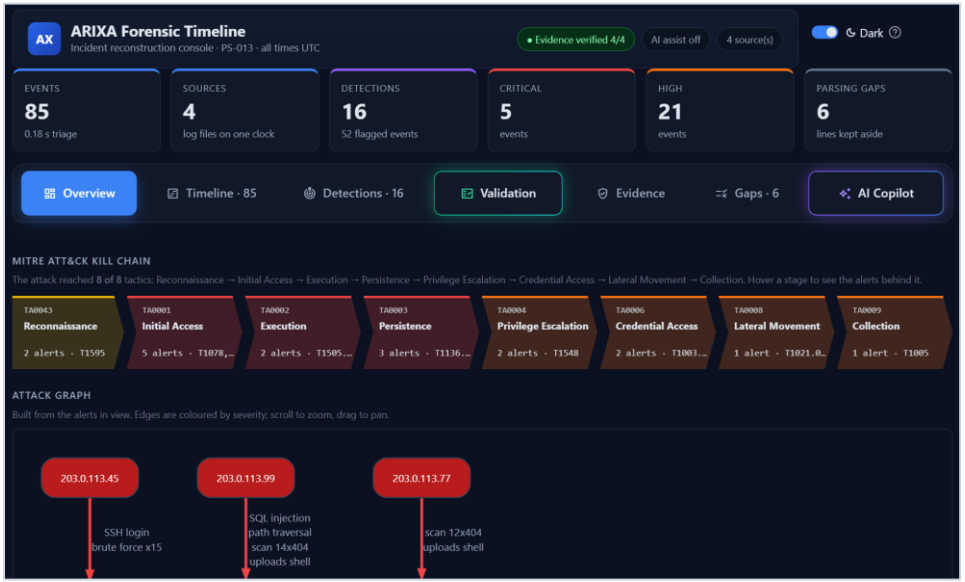
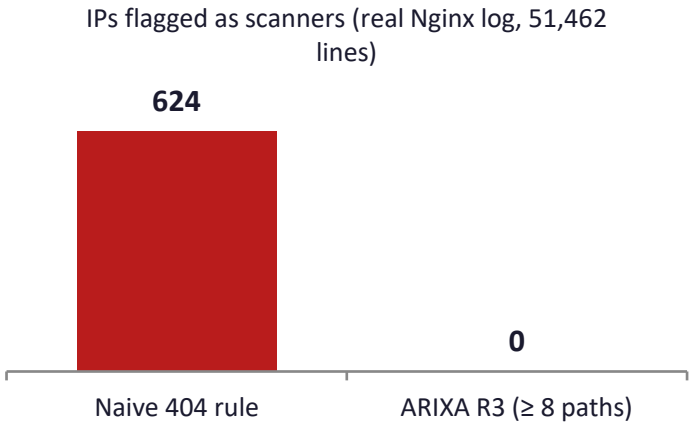


Figure: ARIXA dashboard: KPIs, ATT&CK kill chain (8/8 tactics), attack graph

Optimization Proposed

Before vs after, validated on synthetic incidents with an answer key and on real public logs

Aspect	Before: manual triage	After: ARIXA
Time alignment	Zones converted by hand; clock drift ignored	Automatic UTC: IANA zones, DST, per-source skew; assumptions logged
Evidence integrity	Files edited in place; no record	SHA-256 at ingest, read-only copies, custody log, one-click verify
Completeness	Unreadable lines silently skipped	Every line parsed, or logged as a gap with its reason
Detection	Eyeballing and grep	8 explainable rules, ATT&CK IDs, kill chain, attack graph
Triage time (3 scenarios)	~45 min (team estimate)	0.23 s measured (about 12,000× faster)
Reporting	Copy-paste into a document	Markdown / PDF report; case ZIP with hashes, custody log, CSV



3/3 scenarios passed

16/16 expected alerts

0 false positives

6/6 gaps logged

4/4 hashes verified

14 tests passing

Real-world logs and AI safety

- **LogHub OpenSSH [19]:** 2,000 lines, 10 alerts, 9 brute-force IPs, 5/5 checks
- **Elastic Nginx [21]:** 51,462 lines, 0 false alarms, 5/5 checks
- **AI copilot [20]:** ≤ 60 redacted events, cited IDs verified, custody-logged

Solution Architecture & Design

Layered, modular architecture: presentation, application services and a local data layer; no server required

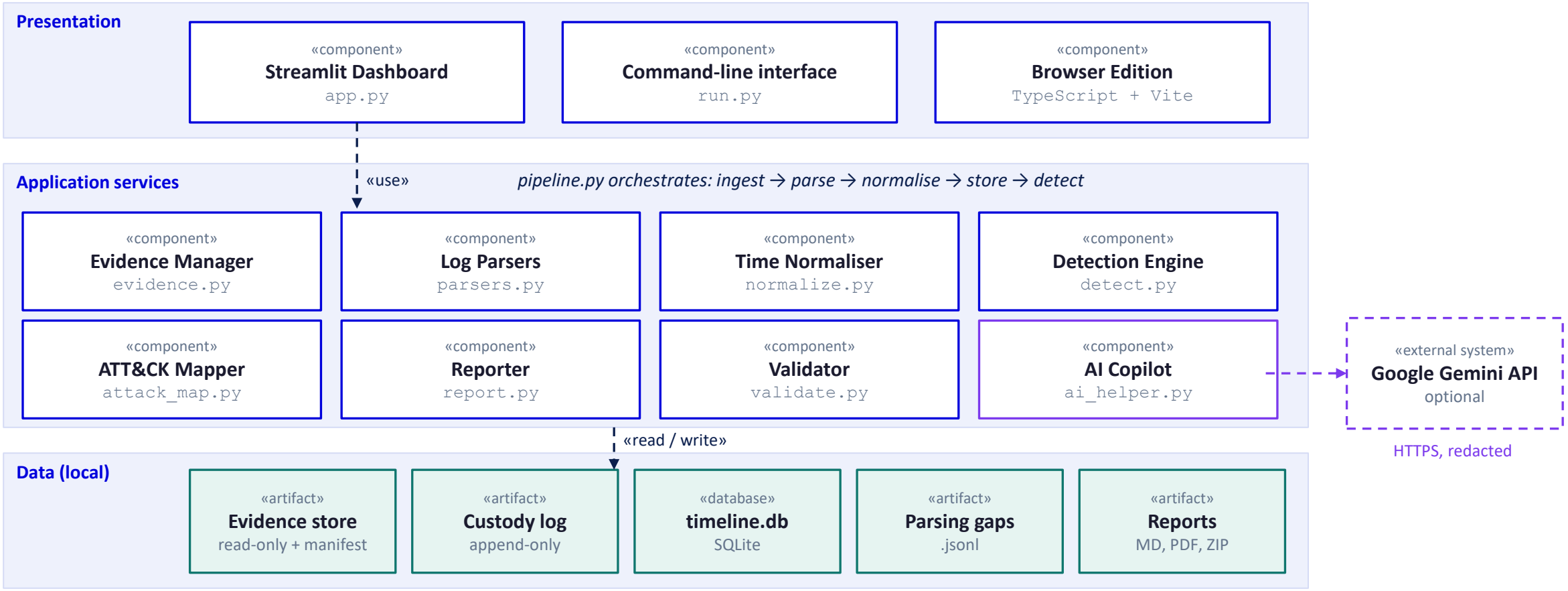


Figure: System Architecture UML Diagram (component view)

Data Flow Diagram

Logs enter read-only, every processing step writes to an auditable data store, and only redacted data can leave

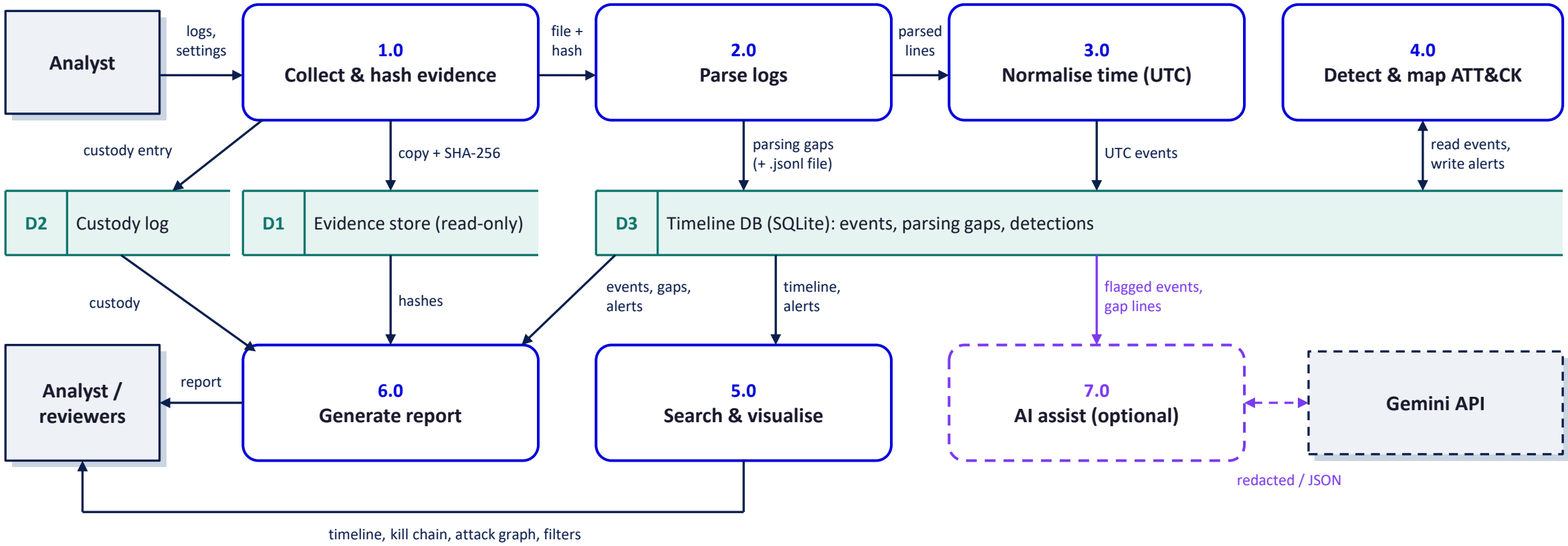


Figure: Data Flow Diagram (DFD), Level 1

Deployment Timeline

Agile delivery in six sprints, each closed by tests and a working demo; two deployment modes, both keep evidence local

Milestone	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5	Sprint 6	Next	Status
M1 Requirements and standards review								Done
M2 Evidence integrity, parsers, gap log								Done
M3 UTC normalisation, SQLite timeline, CLI								Done
M4 Rules R1–R8, kill chain, attack graph								Done
M5 Dashboard, reports, guarded AI copilot								Done
M6 Validation, real logs, browser edition								Done
M7 Demo, documentation, team video								In progress
M8 v2: Windows logs, Sigma rules, auto skew								Planned

Laptop (offline)

```
pip install -r requirements.txt
python run.py demo
streamlit run app.py (port 8080)
```

Browser (zero-install)

Static app on AppDeploy; hashing and analysis run in the browser:
forensic-timeline-showcase-xwfgup.v2.appdeploy.ai

Quality gates

14 pytest tests, validation scorecard, real-world cross-checks, Python ↔ TypeScript parity

Bibliography

Standards, RFCs, research papers, datasets and tools cited in this proposal

[1] K. Kent, S. Chevalier, T. Grance and H. Dang, “Guide to Integrating Forensic Techniques into Incident Response,” NIST SP 800-86, 2006.

[2] P. Cichonski, T. Millar, T. Grance and K. Scarfone, “Computer Security Incident Handling Guide,” NIST SP 800-61 Rev. 2, 2012.

[3] K. Kent and M. Souppaya, “Guide to Computer Security Log Management,” NIST SP 800-92, 2006.

[4] D. Brezinski and T. Killalea, “Guidelines for Evidence Collection and Archiving,” RFC 3227 (BCP 55), IETF, 2002.

[5] ISO/IEC 27037:2012, “Guidelines for identification, collection, acquisition and preservation of digital evidence.”

[6] C. Lonvick, “The BSD syslog Protocol,” RFC 3164, IETF, 2001.

[7] R. Gerhards, “The Syslog Protocol,” RFC 5424, IETF, 2009.

[8] G. Klyne and C. Newman, “Date and Time on the Internet: Timestamps,” RFC 3339, IETF, 2002.

[9] D. Mills, J. Martin, J. Burbank and W. Kasch, “Network Time Protocol Version 4,” RFC 5905, IETF, 2010.

[10] E. Lear and P. Eggert, “Procedures for Maintaining the Time Zone Database,” RFC 6557 (BCP 175), IETF, 2012.

[11] NIST, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015.

[12] R. Fielding, M. Nottingham and J. Reschke, “HTTP Semantics,” RFC 9110 (STD 97), IETF, 2022.

[13] B. E. Strom et al., “MITRE ATT&CK: Design and Philosophy,” The MITRE Corporation, 2018 (rev. 2020).

[14] E. M. Hutchins, M. J. Cloppert and R. M. Amin, “Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains,” 2011.

[15] C. Boyd and P. Forster, “Time and date issues in forensic computing: a case study,” Digital Investigation 1(1), pp. 18–23, 2004.

[16] B. Schatz, G. Mohay and A. Clark, “A correlation method for establishing provenance of timestamps in digital evidence,” Digital Investigation 3S, pp. 98–107, 2006.

[17] C. Hargreaves and J. Patterson, “An automated timeline reconstruction approach for digital forensic investigations,” Digital Investigation 9S, pp. S69–S79, 2012.

[18] K. Gudjonsson, “Mastering the Super Timeline With log2timeline,” SANS Institute, 2010.

[19] J. Zhu, S. He, P. He, J. Liu and M. R. Lyu, “Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics,” IEEE ISSRE, 2023.

[20] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” v2025.

[21] Elastic, “elastic/examples: Common Data Formats, nginx_logs,” GitHub (Apache-2.0).

[22] Google, “Timesketch: collaborative forensic timeline analysis,” GitHub google/timesketch.

[23] log2timeline project, “Plaso: super timeline all the things,” GitHub log2timeline/plaso.



Thank You for Your Attention!

ARIXA Forensic Timeline · A3-PS013-TC479

Incident Timeline and Evidence Collection for Small Teams (PS-013)

Komal Ganjave · Yash Bhosale

Live demo: https://forensic-timeline-showcase-xwfgup.v2.appdeploy.ai/